Manual Técnico — CinemaUSAC (EDD Fase 1)

Universidad de San Carlos de Guatemala

Facultad de Ingeniería — Escuela de Ciencias y Sistemas

Curso: Estructuras de Datos

Proyecto: EDD CinemaUSAC — Fase 1

Estudiante: Cristian Ronaldo Sabán Sequén

Carné: 202303371

Tecnología de interfaz: Qt Widgets (C++), aplicación de escritorio

1. Introducción

Este manual documenta la implementación real del sistema **CinemaUSAC**, contenida en el repositorio `EDD_2S2026_PY_202303371`. El proyecto está desarrollado en **C++ con framework Qt** para la interfaz gráfica, y compuesto por cuatro módulos de backend independientes, cada uno encapsulado en su propia clase, más una clase `MainWindow` que actúa como controlador de la interfaz y conecta los botones de la GUI con la lógica de cada estructura.

Archivos que conforman el proyecto:

Archivo	Contenido
`ArbolPeliculas.h` / `arbolpeliculas.cpp`	Árbol Binario de Búsqueda — Cartelera de películas
`ListaSolicitudes.h` / `listasolicitudes.cpp`	Lista Circular Doblemente Enlazada — Solicitudes especiales
`promociones.h` / `promociones.cpp`	Lista Circular Simple + Listas Doblemente Enlazadas — Promociones y beneficios
`matrizdispersa.h` / `matrizdispersa.cpp`	Matriz Dispersa enlazada — Mapa de asientos
`mainwindow.h` / `mainwindow.cpp`	Controlador de la interfaz gráfica (Qt), conecta la GUI con las 4 estructuras
`main.cpp`	Punto de entrada de la aplicación Qt

Ninguna de las cuatro estructuras utiliza contenedores de la STL (`std::vector` solo se usa como resultado de salida del recorrido Inorden, no como estructura interna); toda la lógica de enlazado, inserción, eliminación y recorrido fue implementada manualmente con punteros crudos (`new` / `delete`).

---

2. Árbol Binario de Búsqueda (BST) — Cartelera de Películas

Clase: `ArbolPeliculas` · **Archivo:** `arbolpeliculas.cpp`

2.1 Estructuras de datos

```
struct Pelicula {
    std::string codigo;
    std::string titulo;
    std::string genero;
    int duracion;
    std::string clasificacion;
    std::string idioma;
    std::string fechaEstreno;
    std::string fechaFin;
};

struct NodoBST {
    Pelicula dato;
    NodoBST* izquierdo;
    NodoBST* derecho;
    NodoBST(Pelicula p) : dato(p), izquierdo(nullptr), derecho(nullptr) {}
};
```

La clase mantiene únicamente un puntero privado `NodoBST* raiz`.

2.2 Operaciones implementadas

- `insertar(Pelicula p)` → `insertarRecursivo`: recorre el árbol comparando `p.codigo` contra `nodo->dato.codigo` como `std::string` (comparación lexicográfica nativa de C++). Si el código ya existe, no se duplica: se imprime un aviso en consola y el árbol queda intacto.
- `eliminar(std::string codigo)` → `eliminarRecursivo`: contempla los tres casos clásicos de un BST:
  1. Nodo sin hijo izquierdo → se reemplaza por su hijo derecho (cubre también el caso hoja).
  2. Nodo sin hijo derecho → se reemplaza por su hijo izquierdo.
  3. Nodo con ambos hijos → se localiza el **sucesor inorden** con `encontrarMinimo(nodo->derecho)` (el nodo más a la izquierda del subárbol derecho), se copia su dato al nodo actual y luego se elimina recursivamente ese sucesor de su posición original.
- `buscarPelicula(std::string codigo)`: búsqueda iterativa aprovechando la propiedad de orden del BST (`O(log n)` en el caso promedio), retorna un puntero a la `Pelicula` encontrada o `nullptr`.
- `existePeliculaPorTitulo(std::string titulo)` → `buscarPorTituloRecursivo`: como el título **no** es la clave de ordenamiento del árbol, esta búsqueda es un recorrido exhaustivo (izquierda y derecha) en `O(n)`. Se usa para validar, al reservar un asiento o crear una función, que la película exista en cartelera.
- `cargarDesdeCSV(ruta)`: lee el archivo con `std::ifstream`, descarta la primera línea (encabezados), y por cada línea usa `std::getline(stream, campo, ',')` para separar los 8 campos del `Pelicula`. La duración se convierte con `std::stoi`, protegido en un bloque `try/catch` que asigna `0` si el valor no es numérico. Cada registro leído se inserta con la misma función `insertar()` usada en el alta manual.
- `obtenerCarteleraInorden()` → `recorridoInorden`: recorre el árbol en **Inorden** (izquierda → raíz → derecha) y acumula los resultados en un `std::vector<Pelicula>`, garantizando que la cartelera se devuelva siempre ordenada ascendentemente por código, tal como lo requiere el enunciado.

- `generarReporteGraphviz()`: recorre el árbol (preorden, dentro de `generarCuerpoDot`) generando un nodo tipo `record` por cada película con código, título, duración y clasificación, y una arista dirigida hacia cada hijo. Escribe el archivo `reporte_cartelera.dot` y ejecuta `system("dot -Tpng reporte_cartelera.dot -o reporte_cartelera.png")`.

2.3 Decisiones de diseño

- Se usó el **código de película como clave del BST** por ser un identificador único de longitud consistente (`P001`, `P002`...), lo que da un ordenamiento predecible y evita ambigüedades por mayúsculas/tildes en los títulos.
- El **recorrido Inorden** se eligió deliberadamente para `obtenerCarteleraInorden()` porque, al insertar por código, este recorrido entrega la cartelera ya ordenada sin necesidad de aplicar un algoritmo de ordenamiento adicional sobre el resultado.
- La búsqueda por título (`existePeliculaPorTitulo`) se mantuvo como una función aparte, separada de la búsqueda por código, porque el título no cumple la propiedad de orden del árbol; se documenta explícitamente en el código que esta búsqueda es lineal, a diferencia de `buscarPelicula` que sí aprovecha la estructura binaria.
- El **estado de cartelera** (próximo estreno / en cartelera / próximo a retirar) y la **alerta de 7 días** no se almacenan en el nodo: se calculan en `MainWindow::on_btnActualizarCartelera_clicked()` comparando `fechaFin` contra `QDate::currentDate()` con `QDate::daysTo()`. Esto evita que la información quede desactualizada si el reporte se consulta en una fecha distinta a la del registro.
- La inserción **ignora duplicados** por código en lugar de sobrescribir el nodo existente, priorizando la integridad de la cartelera sobre la actualización silenciosa de datos.

---

3. Lista Circular Doblemente Enlazada — Solicitudes Especiales

Clase: `ListaSolicitudes` · **Archivo:** `listasolicitudes.cpp`

3.1 Estructuras de datos

```
struct Solicitud {
    int id; // Autogenerado
    std::string cliente;
    std::string telefono;
    std::string tipo;
    std::string descripcion;
    std::string fecha;
    std::string estado; // Pendiente, En proceso, Atendida, Rechazada
};

struct NodoSolicitud {
    Solicitud dato;
    NodoSolicitud* siguiente;
    NodoSolicitud* anterior;
    NodoSolicitud(Solicitud s) : dato(s), siguiente(nullptr), anterior(nullptr) {}
};
```

La clase mantiene `NodoSolicitud* cabeza` y un contador privado `int contadorIds` (inicializado en `1`) para autogenerar el número de solicitud.

3.2 Operaciones implementadas

- `registrarSolicitud(...)`: crea la `Solicitud` con `estado = "Pendiente"` fijo por defecto y `id = contadorIds++`. Si la lista está vacía, el nuevo nodo se autoenlaza (`cabeza->siguiente = cabeza->anterior = cabeza`), cerrando el círculo desde el primer elemento. Si ya hay nodos, se ubica el último con `cabeza->anterior` (propiedad directa de la circularidad doble) y se reconectan los 4 punteros para insertar el nuevo nodo justo antes de la cabeza.
- `procesarPrimeraSolicitud()`: imprime en consola los datos de `cabeza` sin modificar la lista (usada como base conceptual de `obtenerPrimeraSolicitud`, que es la versión que consume la GUI).
- `obtenerPrimeraSolicitud()`: retorna un puntero a `cabeza->dato`, o `nullptr` si la lista está vacía; es la función que `MainWindow::on_btnCargarSolicitud_clicked()` usa para mostrar la solicitud actual en los `QLabel` de la interfaz.
- `aprobarPrimeraSolicitud()`: cambia `cabeza->dato.estado = "Atendida"` y **rota** la lista haciendo `cabeza = cabeza->siguiente`. La solicitud aprobada permanece en la lista (con su estado actualizado), pero deja de estar "al frente" de la cola.
- `rechazarPrimeraSolicitud()`: elimina físicamente el nodo cabeza. Contempla el caso especial de lista con un solo nodo (`cabeza->siguiente == cabeza`, se deja `cabeza = nullptr`) y el caso general, donde se reconecta `ultimo->siguiente` y `cabeza->anterior` saltando el nodo eliminado antes de hacer `delete`.
- `consultarEstado(std::string telefono)`: recorrido circular controlado con `do...while (actual != cabeza)` que compara el teléfono ingresado; retorna un string con el tipo y estado de la solicitud encontrada, o un mensaje de "no encontrada".
- `generarReporteGraphviz()`: primero recorre la lista una vuelta creando un nodo `Mrecord` por cada solicitud (ID, cliente, tipo, estado); luego hace una segunda pasada creando las aristas `actual -> actual->siguiente` con el atributo `dir=both`, representando visualmente el doble enlace. Genera `reporte_solicitudes.dot` → `reporte_solicitudes.png`.

3.3 Decisiones de diseño

- Se usó **doble enlace** porque `rechazarPrimeraSolicitud()` necesita acceso directo al nodo anterior (`cabeza->anterior`) para reconectar el círculo sin tener que recorrer toda la lista buscando al predecesor, lo cual sería obligatorio en una lista simplemente enlazada.
- La **circularidad** permite que `cabeza->anterior` apunte siempre al último nodo, dando acceso en `O(1)` tanto al primero como al último elemento de la cola de atención, sin necesidad de mantener un puntero `cola` adicional.
- **Aprobar vs. Rechazar** se implementaron con comportamientos distintos a propósito: aprobar **conserva** el nodo (cambia estado y rota la cabeza), mientras que rechazar **elimina** el nodo de la lista, en línea con lo indicado en el enunciado del proyecto ("Rechazar la solicitud (Rechazarla de la lista)"). Esto mantiene la lista enfocada en solicitudes que aún requieren seguimiento.
- El **ID es autogenerado** mediante un contador simple (`contadorIds`) en vez de solicitarlo al usuario, evitando colisiones o necesidad de validación de unicidad.
- La fecha de la solicitud se encuentra actualmente fijada como cadena literal (`"2026-08-16"`) en `MainWindow::on_btnEnviarSolicitud_clicked()`; queda documentado como punto de mejora para fases posteriores reemplazarla por `QDate::currentDate()`.

---

4. Lista Circular Simple de Listas — Promociones y Beneficios

Clase: `GestorPromociones` · **Archivo:** `promociones.cpp`

4.1 Estructuras de datos

```
struct Beneficio {
    std::string tipo;
    std::string descripcion;
    std::string valor;
};

struct NodoBeneficio {
    Beneficio dato;
    NodoBeneficio* siguiente;
    NodoBeneficio* anterior;
    NodoBeneficio(Beneficio b) : dato(b), siguiente(nullptr), anterior(nullptr) {}
};

struct Promocion {
    std::string codigo;
    std::string nombre;
    std::string vigencia;
    std::string dias;
};

struct NodoPromocion {
    Promocion dato;
    NodoBeneficio* cabezaBeneficios; // sub-lista doblemente enlazada
    NodoPromocion* siguiente;        // enlace circular simple
    NodoPromocion(Promocion p) : dato(p), cabezaBeneficios(nullptr), siguiente(nullptr) {}
};
```

La clase mantiene `NodoPromocion* cabezaPromociones` como único puntero raíz.

4.2 Operaciones implementadas

- `registrarPromocion(...)`: si `cabezaPromociones` es `nullptr`, el primer nodo se autoenlaza (`siguiente` apunta a sí mismo). En caso contrario, se recorre desde la cabeza hasta encontrar el nodo cuyo `siguiente == cabezaPromociones` (el último de la vuelta) y se inserta el nuevo nodo entre ese último y la cabeza, cerrando nuevamente el círculo.
- `buscarPromocion(std::string codigo)` *(privado)*: recorrido circular acotado con `do...while (actual != cabezaPromociones)` que compara el código; retorna el puntero al nodo o `nullptr`.
- `anadirBeneficio(...)`: localiza la promoción destino con `buscarPromocion`; si no existe, informa el error por consola sin insertar nada. Si existe, inserta el nuevo `NodoBeneficio` **al final** de su sub-lista doblemente enlazada: si la sub-lista está vacía se asigna directamente a `cabezaBeneficios`; si no, se recorre hasta el último nodo (`siguiente == nullptr`) y se enlaza el nuevo nodo con `anterior` apuntando al antiguo último.
- `obtenerTextoPromociones()`: recorre la lista circular de promociones una vuelta y, para cada una, recorre su sub-lista de beneficios, construyendo un `std::string` legible (con viñetas y separadores) que se muestra directamente en un `QMessageBox` desde `MainWindow::on_btnVerPromociones_clicked()`.
- `generarReporteGraphviz()`: construye el archivo `.dot` en tres pasadas:
    1. Agrupa todos los códigos de promoción en la misma fila visual (`rank=same`) para que se vean alineados horizontalmente.
    2. Crea los nodos de promoción (`record` con código, nombre y vigencia) y las aristas circulares entre ellas.

3. Por cada promoción, dibuja su sub-lista de beneficios como una cadena vertical de nodos (`_B0`, `_B1`, ...), con una arista punteada (`dashed`) desde la promoción hacia su primer beneficio, y aristas `dir=both` entre beneficios consecutivos para representar el doble enlace. Genera `reporte_promociones.dot` → `reporte_promociones.png`.

4.3 Decisiones de diseño

- Se implementó como **lista circular simple** en el nivel de promociones (un solo puntero `siguiente`, sin `anterior`) porque el enunciado especifica ese comportamiento exacto y porque las operaciones sobre promociones (registrar, buscar, listar) solo requieren recorrido hacia adelante.
- La sub-lista de beneficios se implementó como **lista doblemente enlazada simple (no circular)**: termina en `nullptr` en lugar de cerrar el círculo, ya que el enunciado pide doble enlace para los beneficios pero no exige que esa sub-lista sea circular; esto simplifica la condición de parada al insertar (`while (actual->siguiente != nullptr)`).
- Todo recorrido de la lista circular de promociones se controla con `do...while (actual != cabezaPromociones)` en vez de comparar contra `nullptr`, ya que en una lista circular ningún nodo apunta naturalmente a `nullptr`; sin este patrón el recorrido caería en un ciclo infinito.
- `anadirBeneficio` valida primero la existencia de la promoción antes de crear el nodo de beneficio, evitando beneficios "huérfanos" que no estén asociados a ninguna promoción válida.
- El reporte Graphviz separa visualmente el **nivel horizontal** (promociones) del **nivel vertical** (beneficios) usando `rank=same` para las promociones, reflejando fielmente el diagrama de referencia del enunciado del proyecto.

---

5. Matriz Dispersa — Mapa de Asientos por Función

Clase: `MatrizDispersa` · **Archivo:** `matrizdispersa.cpp`

5.1 Estructura de datos

```
struct NodoMatriz {
    int fila;
    int columna;
    std::string cliente; // "LIBRE" o el nombre del cliente

    NodoMatriz* arriba;
    NodoMatriz* abajo;
    NodoMatriz* izquierda;
    NodoMatriz* derecha;

    // fila = -1 y columna = -1 se usan para los nodos de cabecera y la raíz
    NodoMatriz(int f, int c, std::string cli = "LIBRE")
        : fila(f), columna(c), cliente(cli),
          arriba(nullptr), abajo(nullptr), izquierda(nullptr), derecha(nullptr) {}
};
```

La clase mantiene un único `NodoMatriz* raiz` (nodo ancla en la posición conceptual (-1,-1)), además de metadatos de la función activa: `peliculaActual`, `horarioActual`, `salaActual`.

5.2 Diseño de la matriz enlazada

La matriz se implementa con el modelo clásico de **matriz dispersa con nodos cabecera de fila y de columna**, todos enlazados en cruz respecto a la raíz:

- `raiz->abajo` encadena las **cabeceras de fila** (ordenadas ascendentemente).
- `raiz->derecha` encadena las **cabeceras de columna** (ordenadas ascendentemente).
- Cada asiento real se enlaza simultáneamente en **cuatro direcciones** (`arriba`, `abajo`, `izquierda`, `derecha`), conectándose tanto a su fila como a su columna correspondiente.

> **Nota importante sobre el alcance de esta implementación:** a diferencia de una matriz dispersa "pura" que solo almacena las celdas con datos relevantes, `inicializarSala()` crea explícitamente **un nodo por cada asiento de la sala** (incluyendo los libres, con `cliente = "LIBRE"`), recorriendo todas las combinaciones fila×columna. Esto se hizo así para poder pintar el mapa completo de la sala en la tabla de la interfaz (`on_btnRefrescarSala_clicked`) consultando directamente cada posición. La estructura conserva, no obstante, el diseño de nodos cabecera cruzados propio de una matriz dispersa enlazada, tal como lo pide el enunciado.

5.3 Operaciones implementadas

- `buscarFila(int fila)` / `buscarColumna(int columna)`: recorren `raiz->abajo` / `raiz->derecha` linealmente comparando el número de fila/columna.
- `crearFila(int fila)` / `crearColumna(int columna)`: insertan una nueva cabecera **en orden ascendente**, recorriendo hasta encontrar la posición correcta y reconectando los punteros `arriba`/`abajo` (para filas) o `izquierda`/`derecha` (para columnas).
- `insertarAsientoLibre(fila, columna)` *(privado)*: localiza o crea la cabecera de fila y de columna correspondientes, crea el `NodoMatriz` del asiento en estado `"LIBRE"`, y lo enlaza en dos pasos independientes:
  1. Inserción horizontal dentro de la lista de la fila (mantiene orden por columna).
  2. Inserción vertical dentro de la lista de la columna (mantiene orden por fila).
- `inicializarSala(filas, columnas, pelicula, horario, sala)`: guarda los metadatos de la función y ejecuta un doble ciclo `for` que invoca `insertarAsientoLibre` por cada combinación fila×columna, construyendo la matriz completa desde cero.
- `reservarAsiento(fila, columna, cliente)`: busca la cabecera de fila; si no existe, retorna `false`. Recorre la fila hacia la derecha comparando columnas; si encuentra el asiento y está `"LIBRE"`, lo marca con el nombre del cliente y retorna `true`. Si ya está ocupado, informa por consola quién lo ocupa y retorna `false` sin modificar nada.
- `cancelarReserva(fila, columna)`: misma lógica de búsqueda que `reservarAsiento`; si el asiento está ocupado, lo regresa a `"LIBRE"`.
- `cancelarPorCliente(std::string nombreCliente)`: recorre **toda** la matriz (todas las filas, y dentro de cada una todas las columnas) liberando cualquier asiento cuyo `cliente` coincida con el nombre buscado; es la función que usa la GUI para la cancelación de reservas desde la vista de cliente (que solo pide el nombre, no fila/columna).
- `obtenerEstadoAsiento(fila, columna)`: retorna el string `cliente` del nodo encontrado (`"LIBRE"` o el nombre de quien reservó), o `"ND"` (No Disponible) si la fila o la columna no existen. Es la función que alimenta la tabla visual de asientos en `on_btnRefrescarSala_clicked()`.
- `generarReporteGraphviz()`: construye el `.dot` en varias etapas: nodo raíz con los metadatos de la función, cabeceras de columna alineadas en la misma fila (`rank=same`) con doble flecha entre ellas, y luego, por cada fila, su cabecera y todos sus asientos agrupados con `rank=same`, coloreados en verde (`lightgreen`) si están libres o salmón (`salmon`) si están ocupados, con aristas `dir=both` conectándolos tanto a su vecino izquierdo como al asiento de arriba (o a la cabecera de columna, si es el primero de

esa columna). Al final agrega un nodo de estadísticas (`Stats`) con el total de asientos, libres y ocupados. Genera `reporte_matriz.dot` → `reporte_matriz.png`.

5.4 Decisiones de diseño

- Se optó por el modelo de **matriz dispersa con cabeceras cruzadas de fila y columna** (en vez de una tabla hash o un vector de pares) porque es el modelo estándar enseñado en el curso para esta estructura, y porque el reporte Graphviz exigido en el enunciado (Reporte 4) describe explícitamente nodos cabecera de fila y columna conectados a los nodos de asiento.
- Se decidió **crear un nodo por cada asiento** (incluidos los libres) en lugar de omitir los libres, priorizando la simplicidad de las consultas de estado (`obtenerEstadoAsiento`) y el pintado completo de la tabla en la interfaz, a costa de no aprovechar al máximo el ahorro de memoria que ofrecería omitir los nodos libres. Esta decisión es coherente con la nota de alcance de Fase 1 del enunciado, que limita el sistema a una sola función activa a la vez.
- La búsqueda dentro de una fila/columna se detiene tan pronto `actual->columna > columna` (o el equivalente en fila), aprovechando que los nodos están **ordenados** dentro de cada lista, aunque en la práctica, al crearse todos los asientos secuencialmente, esto no cambia la complejidad de forma significativa.
- `cancelarPorCliente` se implementó como búsqueda exhaustiva de toda la matriz porque el enunciado indica que el cliente cancela ingresando su **nombre**, no la fila y columna exacta, replicando el flujo de "Cancelar Reserva" descrito para el usuario final.
- Al **crear una nueva función** (`on_btnCrearFuncion_clicked`), la matriz anterior se descarta reemplazando el objeto completo con `sala = MatrizDispersa();` antes de llamar a `inicializarSala(...)`, cumpliendo la nota explícita del enunciado de que en Fase 1 solo existe una función activa a la vez y la anterior se sobreescribe. Lo mismo ocurre en `on_btnEliminarFuncion_clicked()`.

---

6. Controlador de Interfaz — `MainWindow`

Archivos: `mainwindow.h`, `mainwindow.cpp`, `mainwindow.ui`

6.1 Rol del controlador

`MainWindow` no contiene lógica de estructuras de datos; su responsabilidad es exclusivamente **traducir eventos de la interfaz Qt** (clics de botones) en llamadas a los métodos públicos de las cuatro clases de backend, y reflejar sus resultados en widgets (`QTableWidget`, `QLabel`, `QMessageBox`).

Las cuatro estructuras se instancian como **miembros persistentes** de la ventana:

```
ArbolPeliculas cartelera;
ListaSolicitudes solicitudes;
MatrizDispersa sala;
GestorPromociones promociones;
```

Esto asegura que el estado de cada estructura se mantenga en memoria mientras la aplicación esté abierta, sin necesidad de persistencia en disco (no requerida en esta fase).

6.2 Navegación y sesión

La aplicación usa un `QStackedWidget` (`ui->B`) con tres páginas: `page1Login`, `page2Admin` y `page3Usuario`. El inicio de sesión (`on_btnIniciarSesion_clicked`) valida credenciales fijas en código (`admin@usac.edu.gt` / `cliente@gmail.com`) y cambia de página según el rol autenticado; no existe todavía persistencia de usuarios ni cifrado de contraseñas, consistente con el alcance de Fase 1.

6.3 Flujos principales conectados a cada estructura

Botón / Slot	Estructura	Acción
`on_btnAgregarPelicula_clicked`	BST	`cartelera.insertar(...)`
`on_btnCargaMasiva_clicked`	BST	Abre diálogo de archivo (`QFileDialog`) y llama `cartelera.cargarDesdeCSV(...)`
`on_btnActualizarCartelera_clicked`	BST	Llena tabla admin + calcula alertas de 7 días con `QDate`
`on_btnBuscarPelicula_clicked`	BST	`cartelera.buscarPelicula(...)`
`on_btnVerCarteleraUsuario_clicked`	BST	Llena tabla de cliente con `obtenerCarteleraInorden()`
`on_btnEnviarSolicitud_clicked`	Lista circular doble	`solicitudes.registrarSolicitud(...)`
`on_btnCargarSolicitud_clicked`	Lista circular doble	Muestra la solicitud en `cabeza` vía `obtenerPrimeraSolicitud()`
`on_btnAprobarSolicitud_clicked`	Lista circular doble	`solicitudes.aprobarPrimeraSolicitud()`
`on_btnRechazarSolicitud_clicked`	Lista circular doble	`solicitudes.rechazarPrimeraSolicitud()`
`on_btnConsultarEstado_clicked`	Lista circular doble	`solicitudes.consultarEstado(telefono)`
`on_btnCrearPromo_clicked`	Lista circular simple	`promociones.registrarPromocion(...)`
`on_btnAnadirBeneficio_clicked`	Lista doble (sub-lista)	`promociones.anadirBeneficio(...)`

Botón / Slot	Estructura	Acción
`on_btnVerPromociones_clicked`	Lista circular simple	`promociones.obtenerTextoPromociones()`
`on_btnCrearFuncion_clicked`	Matriz dispersa	Valida película existente y llama `sala.inicializarSala(...)`
`on_btnReservar_clicked`	Matriz dispersa	Valida película existente y llama `sala.reservarAsiento(...)`
`on_btnCancelarReserva_clicked`	Matriz dispersa	`sala.cancelarPorCliente(...)`
`on_btnRefrescarSala_clicked`	Matriz dispersa	Pinta la tabla de asientos consultando `sala.obtenerEstadoAsiento(f,c)` por cada celda
`on_btnEliminarFuncion_clicked`	Matriz dispersa	Reinicia `sala` con `sala = MatrizDispersa();`
`on_btnReportes_clicked`	Las 4 estructuras	Genera los 4 reportes Graphviz de una sola vez
`on_btnVerRepCartelera/Solicitudes/Sala/Promos_clicked`	Cada estructura	Regenera el reporte individual y lo carga en `QLabel` con `QPixmap`

6.4 Decisiones de diseño relevantes

- Las **validaciones de consistencia** (por ejemplo, no permitir reservar un asiento o crear una función para una película que no existe en la cartelera) se implementaron en el controlador, llamando a `cartelera.existePeliculaPorTitulo(...)` antes de tocar la matriz de asientos. Esto evita que la matriz dispersa tenga referencias a películas inexistentes, manteniendo la integridad referencial entre el BST y la matriz.
- Los reportes se **regeneran bajo demanda** justo antes de mostrarse (por ejemplo, `on_btnVerRepCartelera_clicked` llama `cartelera.generarReporteGraphviz()` inmediatamente antes de cargar la imagen), garantizando que el `.png` mostrado refleje siempre el estado más reciente de la estructura, en línea con el requerimiento de "visualización en tiempo real" del enunciado.
- Se usa una función auxiliar libre `cargarImagenEnVisor(QLabel*, QString)` para evitar duplicar la lógica de carga de `QPixmap` en los cuatro visores de reporte.

---

7. Módulo de Reportes con Graphviz — Resumen técnico

Cada una de las cuatro clases de backend implementa su propio método `generarReporteGraphviz()`, totalmente independiente de las demás, siguiendo el mismo patrón:

1. Abrir un `std::ofstream` hacia un archivo `.dot` con nombre fijo (`reporte_cartelera.dot`, `reporte_solicitudes.dot`, `reporte_promociones.dot`, `reporte_matriz.dot`).
2. Recorrer la estructura correspondiente escribiendo la sintaxis DOT (nodos y aristas) directamente como texto.
3. Cerrar el archivo y ejecutar `system("dot -Tpng archivo.dot -o archivo.png")` para invocar el binario de Graphviz instalado en el sistema.
4. La interfaz carga el `.png` resultante en un `QLabel` mediante `QPixmap`.

Esta arquitectura modular permite regenerar el reporte de una sola estructura sin afectar a las demás, cumpliendo con el requerimiento de programación modular del enunciado.

---

8. Manejo de Memoria

El proyecto utiliza exclusivamente memoria dinámica manual (`new` / `delete`), sin punteros inteligentes ni contenedores STL para las estructuras principales. Puntos relevantes detectados en el código actual:

- `ArbolPeliculas::eliminarRecursivo` libera correctamente cada nodo eliminado con `delete nodo` (o `delete` implícito al reasignar en el caso de dos hijos, ya que se elimina el sucesor).
- `ListaSolicitudes::rechazarPrimeraSolicitud` libera el nodo removido con `delete aBorrar`.
- `MatrizDispersa`, al reemplazarse con `sala = MatrizDispersa();` en `on_btnCrearFuncion_clicked` y `on_btnEliminarFuncion_clicked`, **no libera explícitamente** los nodos de la matriz anterior (filas, columnas y asientos) antes de perder la referencia a `raiz`; esto representa una fuga de memoria pendiente de resolver en una fase de refactorización, ya que la clase no implementa actualmente un destructor (`~MatrizDispersa()`) que recorra y libere todos sus nodos.
- De manera similar, `GestorPromociones` y `ArbolPeliculas` tampoco implementan un destructor que libere toda la estructura al finalizar el programa; dado que el sistema es de corta duración (una sola sesión de uso), el sistema operativo libera esa memoria al cerrar la aplicación, pero se documenta como mejora recomendada para fases posteriores.

---

9. Conclusiones

La implementación real del proyecto respeta el diseño exigido en el enunciado para las cuatro estructuras de datos:

- El **BST** ordena la cartelera por código y expone tanto búsqueda binaria (por código) como búsqueda lineal (por título), usando el recorrido Inorden para listar la cartelera completa ordenada.
- La **lista circular doblemente enlazada** de solicitudes aprovecha el doble enlace para aprobar (rotar) y rechazar (eliminar) sin recorridos adicionales, manteniendo acceso O(1) a la cabeza y la cola mediante `cabeza->anterior`.
- La **lista circular simple de listas** modela correctamente la relación 1:N entre promociones y beneficios, separando el nivel circular (promociones) del nivel doblemente enlazado no circular (beneficios).
- La **matriz dispersa** usa el modelo clásico de nodos cabecera de fila/columna con enlaces cruzados, aunque en esta implementación se materializan también los asientos libres para simplificar el pintado de la interfaz, decisión documentada como una desviación consciente del ahorro máximo de memoria que ofrecería una matriz dispersa estricta.

Cada estructura cuenta con su módulo independiente de generación de reportes en Graphviz, y toda la interacción con el usuario se centraliza en la clase `MainWindow`, que actúa exclusivamente como capa de presentación sobre las cuatro estructuras de backend.